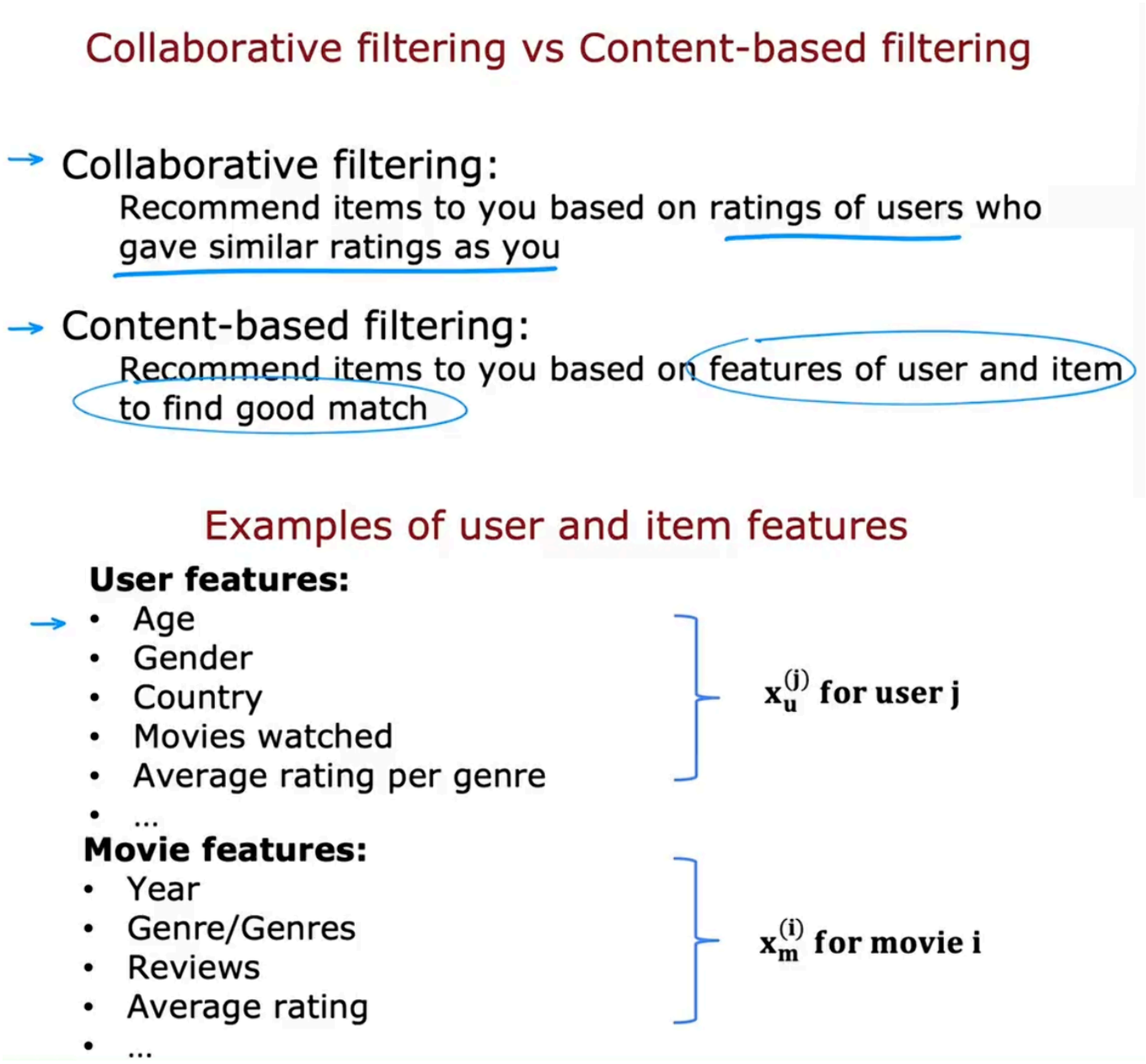
Day 32. Content-based filtering

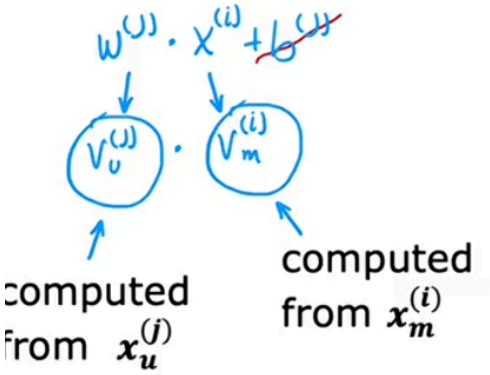
Content-based filtering

Content-based filtering vs. Collaborative filtering



Content-based filtering: Learning to match

Predict rating of user j on movie i as

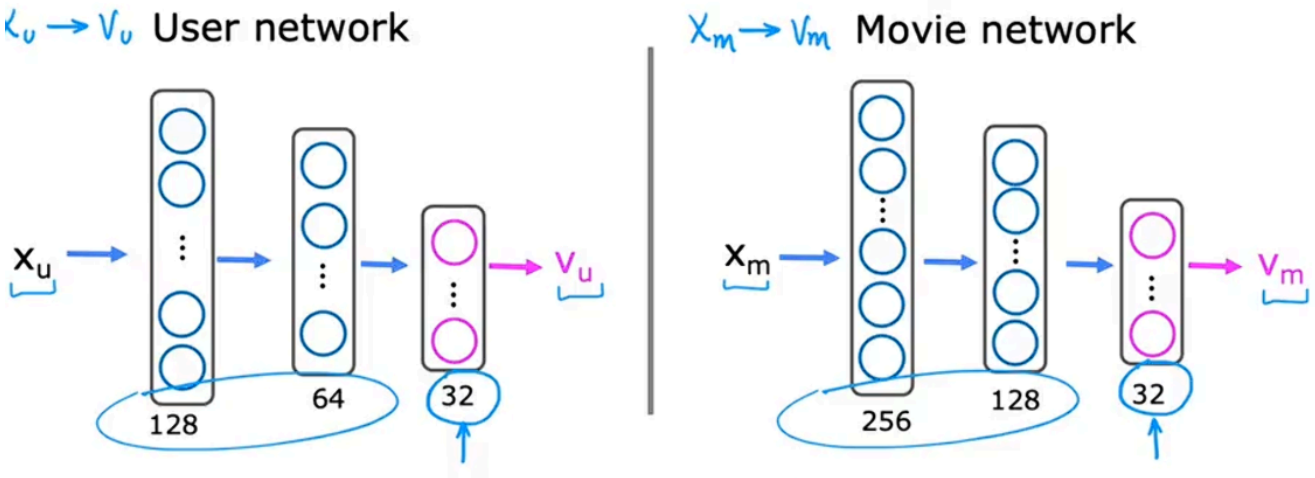


user's preferences $v_u = \begin{bmatrix} 4.9 \\ 0.1 \\ \vdots \\ 3.0 \end{bmatrix}$ likes, likes

movie features $v_m = \begin{bmatrix} 4.5 \\ 0.2 \\ \vdots \\ 3.5 \end{bmatrix}$ romance, action

Deep learning for content-based filtering

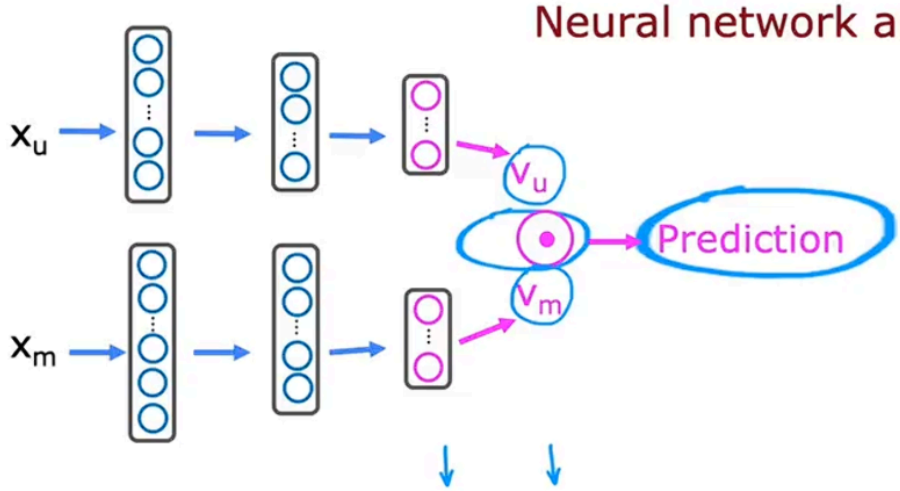
Neural network architecture



Prediction : $v_u^{(j)} \cdot v_m^{(i)}$

$g\left(v_u^{(j)} \cdot v_m^{(i)}\right)$ ***to predict the probability that*** $y^{(i,j)}$ ***is*** 1

Neural network architecture



Cost function $J = \sum_{(i,j):r(i,j)=1} \left(v_u^{(j)} \cdot v_m^{(i)} - y^{(i,j)}\right)^2$ + NN regularization term

Learned user and item vectors:

$v_u^{(j)}$ is a vector of length 32 that describes user j with features $x_u^{(j)}$

$v_m^{(i)}$ is a vector of length 32 that describes movie i with features $x_m^{(i)}$

To find movies similar to movie i: $\|v_m^{(k)} - v_m^{(i)}\|^2$ small

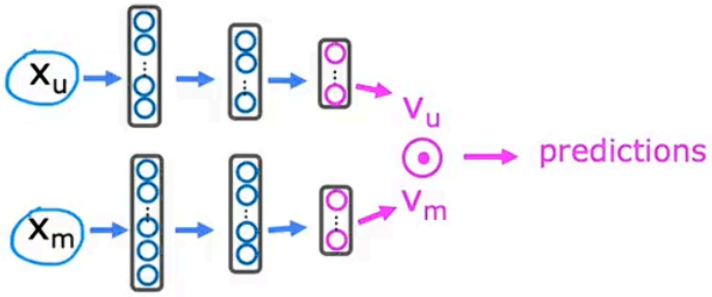
$\|x^{(k)} - x^{(i)}\|^2$

Note: This can be pre-computed ahead of time

Recommending from a large catalogue

How to efficiently find recommendation from a large set of items?

- Movies 1000+
- Ads 1m+
- Songs 10m+
- Products 10m+



Two steps: Retrieval & Ranking

Retrieval:

- Generate large list of plausible item candidates

  e.g.

  1) For each of the last 10 movies watched by the user, find 10 most similar movies

  2) For most viewed 3 genres, find the top 10 movies

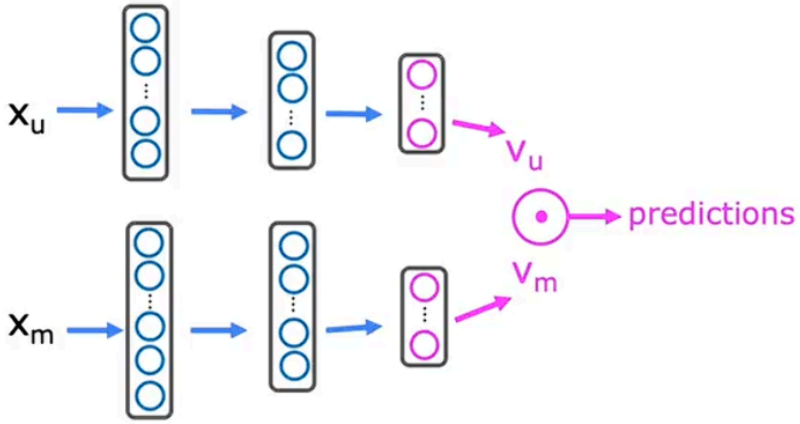
  3) Top 20 movies in the country

- Combine retrieved items into list, removing duplicates and items already watched/purchased

Two steps: Retrieval & ranking

Ranking: ~100s

- Take list retrieved and rank using learned model



- Display ranked items to user

Retrieval step

- Retrieving more items results in better performance, but slower recommendations.
- To analyse/optimize the trade-off, carry out offline experiments to see if retrieving additional items results in more relevant recommendations (i.e., $p(y^{(i,j)}) = 1$ of items displayed to user are higher).

Ethical use of recommendation system

What is the goal of the recommender system?

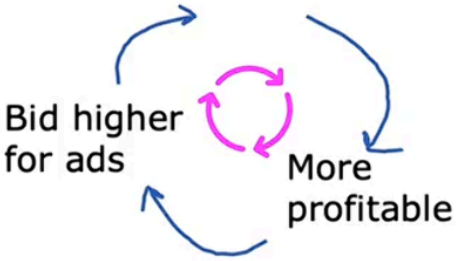
Recommend:

- Movies most likely to be rated 5 stars by user
- Products most likely to be purchased
- Ads most likely to be clicked on
- Products generating the largest profit
- Video leading to maximum watch time

Ethical considerations with recommender systems

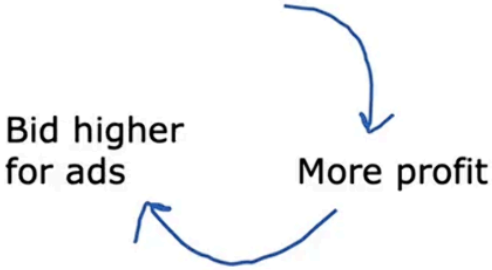
Travel industry

Good travel experience to more users



Payday loans

Squeeze customers more



Other problematic cases:

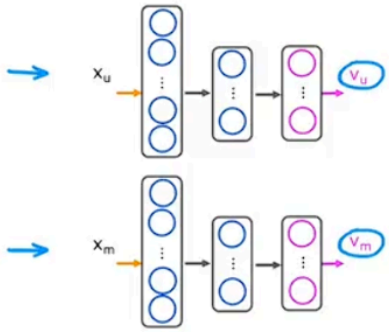
- Maximizing user engagement (e.g. watch time) has led to large social media/video sharing sites to amplify conspiracy theories and hate/toxicity

  Amelioration : Filter out problematic content such as hate speech, fraud, scams and violent content

- Can a ranking system maximize your profit rather than users' welfare be presented in a transparent way?

  Amelioration : Be transparent with users

TensorFlow implementation of content-based filtering




```
user_NN = tf.keras.models.Sequential([
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(32)
])
```

```
item_NN = tf.keras.models.Sequential([
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(32 )
])
```

```
# create the user input and point to the base network
input_user = tf.keras.layers.Input(shape=(num_user_features))
vu = user_NN(input_user)
vu = tf.linalg.l2_normalize(vu, axis=1)

# create the item input and point to the base network
input_item = tf.keras.layers.Input(shape=(num_item_features))
vm = item_NN(input_item)
vm = tf.linalg.l2_normalize(vm, axis=1)

# measure the similarity of the two vector outputs
output = tf.keras.layers.Dot(axes=1)([vu, vm])

# specify the inputs and output of the model
model = Model([input_user, input_item], output)

# Specify the cost function
cost_fn = tf.keras.losses.MeanSquaredError()
```